

MSSV : 23127195

Họ và tên : Trần Mạnh Hùng

Câu 1 (4.0 điểm) — Phê bình thiết kế hiện tại

Dùng các khái niệm **coupling/cohesion + SOLID + DRY/KISS/YAGNI** để phê bình thiết kế hiện tại.

Yêu cầu

Em phải nêu **ít nhất 8 nhận định**. Với **mỗi nhận định**, phải có đủ 4 ý:

1. **Bằng chứng cụ thể trong case**
2. **Khái niệm liên quan** (coupling/cohesion, SOLID, DRY, KISS, YAGNI)
3. **Hậu quả thực tế nếu hệ thống mở rộng**
4. **Một phản biện ngược lại**: trong bối cảnh hẹp nào thiết kế này vẫn có thể tạm chấp nhận được?

Ràng buộc bắt buộc

- Ít nhất **2 nhận định** phải thuộc nhóm **coupling/cohesion**
- Ít nhất **3 nhận định** phải thuộc nhóm **SOLID**
- Ít nhất **1 nhận định** phải gắn với **DRY/KISS/YAGNI**
- Ít nhất **2 nhận định** phải liên hệ trực tiếp đến các thay đổi mới trong đề bài, ví dụ: thêm Telegram, thêm payment, web/mobile/API

Lưu ý

Không chấp nhận kiểu trả lời như: “vi phạm SRP vì class làm nhiều việc” nếu em không chỉ rõ **làm nhiều việc gì** và **vì sao đó là vấn đề trong context này**.

GIẢI

NĐ1. Tính kết dính thấp (Low cohesion)

- **Bằng chứng cụ thể** : Lớp EventManager vừa quản lý vòng đời sự kiện (CRUD), vừa xử lý đăng ký (Register), vừa thực hiện kiểm soát vé (checkInByQr), lại làm luôn cả việc xuất báo cáo tài chính (generateFinanceReport), các công việc trong đây không liên

quan chặt chẽ đến nhau và có dấu hiệu của god class (tức một class làm quá nhiều việc cùng lúc

- **Khái niệm liên quan:** Cohesion (Tính kết dính).
- **Hậu quả thực tế:** Khi hệ thống mở rộng, ví dụ cần thay đổi định dạng báo cáo từ Excel sang PDF, lập trình viên buộc phải sửa trực tiếp trong lớp EventManager để thêm logic tạo PDF hoặc chỉnh sửa phương thức exportExcelReport. Điều này làm cho một thay đổi ở chức năng báo cáo lại ảnh hưởng đến lớp đang chứa cả logic đăng ký sự kiện và check-in, làm tăng nguy cơ gây lỗi ngoài ý muốn. Ngoài ra, class ngày càng phình to, khiến việc bảo trì, kiểm thử và phân chia công việc giữa các lập trình viên trở nên khó khăn.
- **Phản biện ngược lại:** Thiết kế này có thể chấp nhận được trong giai đoạn làm Prototype cực nhanh (MVP) với duy nhất một lập trình viên, nơi tốc độ viết code quan trọng hơn cấu trúc bền vững.

NĐ2. Phụ thuộc chặt chẽ vào cơ sở dữ liệu (Tight Coupling)

- **Bằng chứng cụ thể:** Dòng code `public static Database db = new MySQLDatabase();` khởi tạo trực tiếp một đối tượng cụ thể của MySQL.
- **Khái niệm liên quan:** Coupling (Độ phụ thuộc).
- **Hậu quả thực tế:** nếu hệ thống muốn chuyển từ MySQL sang PostgreSQL, lập trình viên phải quay lại sửa trực tiếp trong EventManager để thay đổi cách khởi tạo database. Ngoài ra, khi viết Unit Test cho chức năng đăng ký, code có thể buộc phải kết nối tới MySQL thật thay vì dùng database giả, khiến việc kiểm thử trở nên phức tạp và tốn thời gian.
- **Phản biện ngược lại** Nếu hệ thống cực kỳ nhỏ và nhà trường cam kết chỉ sử dụng duy nhất một loại DB trong suốt vòng đời sản phẩm, việc khai báo trực tiếp sẽ giúp code ngắn gọn, giảm bớt các lớp interface trung gian và có thể tập trung viết các business logic thay vì chăm chú vô việc cố gắng mở rộng được nhiều DB.

NĐ3: Vi phạm nguyên lý đơn nhiệm (SRP)

- **Bằng chứng cụ thể:** Phương thức register đang gánh vác quá nhiều trách nhiệm: kiểm tra số chỗ trống (`e.availableSeats > 0`) , lưu vào DB (`db.saveRegistration(s, e)`) , và quyết định gửi thông báo qua 3 kênh khác nhau (`new SmsSender()`, `new PushSender()`; `EmailSender()`)
- **Khái niệm liên quan:** SOLID - Single Responsibility Principle (SRP)
- **Hệ quả trực tiếp :** Khi hệ thống cần sửa chữa một trong 3 trách nhiệm ví dụ như chỉ gửi SMS cho sự kiện nào đó. Thì ta lại phải sửa logic trong hàm register, hoặc như nếu thay đổi logic kiểm tra chỗ trống (`e.availableSeat > 0`) thành 1 chẳng hạn thì lại phải sửa ở đó , càng ngày chúng ta cứ vào sửa register , dễ gây lỗi cho logic vốn dĩ đã đúng
- **Phản biện ngược lại :** Trong các dự án nhỏ hoặc prototype, việc đặt toàn bộ logic đăng ký trong một phương thức giúp lập trình viên dễ theo dõi luồng xử lý từ đầu đến cuối mà không phải chia nhỏ thành nhiều lớp phức tạp.

NĐ4. Vi phạm nguyên lý Mở/Đóng khi thêm Telegram (OCP)

- **Bằng chứng cụ thể:** Cấu trúc hàm register sử dụng các tham số boolean (`sendEmail`, `sendSms`, `sendPush`) và các khối if cứng nhắc để gửi thông báo.
- **Khái niệm liên quan:** SOLID - Open/Closed Principle (OCP).
- **Hậu quả thực tế:** Khi yêu cầu thêm Telegram xuất hiện trong học kỳ tới, ta bắt buộc phải sửa hàm thêm một khối if mới vào giữa hàm register. Điều này phá vỡ tính ổn định của mã nguồn cũ đã được kiểm chứng (tested code) và ta lại phải test lại tất cả các phương thức đã đúng khác gây mất thời gian.
- **Phản biện ngược lại:** Nếu số lượng kênh thông báo là cố định và không bao giờ thay đổi, việc sử dụng các cờ boolean đơn giản là cách làm nhanh và trực quan nhất.

NĐ5. Vi phạm nguyên lý đảo ngược phụ thuộc (DIP)

- **Bằng chứng cụ thể:** EventManager khởi tạo trực tiếp các đối tượng EmailSender, SmsSender, PushSender bằng từ khóa new bên trong phương thức.
- **Khái niệm liên quan:** SOLID - Dependency Inversion Principle (DIP).
- **Hậu quả thực tế:** Lớp mức cao (EventManager) đang phụ thuộc trực tiếp vào các lớp mức thấp (các lớp Sender). Điều này khiến ta không thể thay thế các dịch vụ gửi tin (ví dụ: đổi từ dịch vụ gửi Email A sang dịch vụ B) mà không sửa code của EventManager. Khiến cho class EventManager bị phụ thuộc trực tiếp vào các class Sender
- **Phản biện ngược lại:** Với một ứng dụng nội bộ nhỏ, không có yêu cầu thay đổi nhà cung cấp dịch vụ và không ưu tiên viết Unit Test tự động, việc khởi tạo trực tiếp giúp giảm bớt sự phức tạp.

NĐ6. Lặp lại logic thông báo (Violation of DRY)

- **Bằng chứng cụ thể:** Logic khởi tạo sender và gọi hàm send() lặp lại gần như y hệt cho Email, SMS và Push Notification trong hàm register.
- **Khái niệm liên quan:** DRY (Don't Repeat Yourself).
- **Hậu quả thực tế:** Nếu sau này hệ thống yêu cầu tất cả các thông báo đều phải được ghi log (ghi nhật ký) trước khi gửi, bạn sẽ phải copy đoạn mã ghi log đó vào cả 3 khối if. Càng nhiều kênh thông báo (như Telegram sắp tới), mã nguồn sẽ càng trở nên dư thừa và khó quản lý lỗi đồng bộ.
- **Phản biện ngược lại:** Đôi khi việc lặp lại khiến code dễ đọc hơn là cố gắng trừu tượng hóa quá mức khiến người mới khó tìm ra code thực tế nằm ở đâu.

NĐ7. Bế tắc khi thêm tính năng thanh toán

- **Bằng chứng:** Hàm register mặc định giảm chỗ trống (e.availableSeats--) ngay lập tức và lưu luôn vào DB.
- **Khái niệm: Open-Closed Principle (OCP) – SOLID .**
- **Hậu quả:** Với sự kiện có phí, trình tự đúng phải là: *Giữ chỗ tạm thời -> Thanh toán -> Xác nhận đăng ký*. Code hiện tại không có chỗ để chèn logic thanh toán vào giữa, dẫn đến việc sinh viên có thể đăng ký thành công mà không cần trả tiền.
- **Phản biện:** Chấp nhận được nếu ban đầu CTO chỉ yêu cầu làm cho sự kiện miễn phí. Việc xây dựng một luồng thanh toán phức tạp ngay từ đầu sẽ bị coi là over-engineering theo đúng tinh thần YAGNI.

NĐ8 .Thiết kế không sử dụng (Vi phạm YAGNI)

- **Bằng chứng cụ thể:** Mặc dù chỉ yêu cầu gửi thông báo khi đăng ký thành công, nhưng class lại chuẩn bị sẵn cả `updateEvent` và `deleteEvent` trong cùng một khối xử lý đăng ký.
- **Khái niệm liên quan:** YAGNI (You Ain't Gonna Need It).
- **Hậu quả thực tế:** Việc nhồi nhét mọi chức năng vào một nơi làm tăng độ phức tạp không cần thiết cho ứng dụng **Web/Mobile API**. Khi hệ thống lớn lên, việc bảo trì một class "vạn năng" như thế này tốn nhiều chi phí hơn nhiều so với việc tách nhỏ chúng ngay từ đầu theo từng Module riêng biệt.
- **Phản biện ngược lại:** Có thể hiểu đây là nỗ lực của lập trình viên để "đi trước một bước", chuẩn bị sẵn các hàm CRUD cơ bản để Ban tổ chức có thể dùng ngay khi cần mà không phải yêu cầu code thêm lần nữa.

Câu 2 (3.5 điểm) — Thiết kế lại hệ thống

Thiết kế lại hệ thống bằng:

- Module decomposition
- C4 Context Diagram
- C4 Container Diagram
- 1 UML Sequence Diagram

Yêu cầu chi tiết

2.1. Module decomposition

Đề xuất cách chia hệ thống thành các module/subsystem hợp lí. Cần nêu rõ:

- Module nào chịu trách nhiệm chính cho phần nào
- Ranh giới giữa business logic và infrastructure ở đâu
- Cách chia này cải thiện điều gì so với thiết kế cũ

2.2. C4 Context Diagram

Vẽ sơ đồ Context thể hiện:

- Người dùng chính
- Hệ thống chính

- External systems liên quan

2.3. C4 Container Diagram

Vẽ sơ đồ Container tối thiểu gồm:

- Web App
- Mobile App
- API Application
- Database
- Các external services phù hợp

2.4. UML Sequence Diagram

a. Vẽ 1 sequence diagram cho đúng 1 luồng quan trọng do em tự chọn, ví dụ:

- Đăng ký sự kiện và gửi thông báo
- Đăng ký sự kiện có thanh toán
- Check-in bằng QR

Ràng buộc tư duy phản biện

Sau mỗi phần trong câu 2, em phải viết ngắn gọn:

- Một quyết định em chọn
 - Một phương án khác em đã cân nhắc nhưng loại bỏ
 - Lý do loại bỏ phương án đó ở giai đoạn hiện tại
- b. CTO yêu cầu “đừng over-engineering”, em sẽ không làm điều gì trong thiết kế của mình, và vì sao?

Giải

1. Module Decomposition (Phân rã Mô-đun)

Hệ thống sẽ được chia thành 4 mô-đun lõi, mỗi mô-đun chỉ chịu trách nhiệm cho một miền nghiệp vụ duy nhất:

Module	Trách nhiệm chính
Event Module	Quản lý thông tin sự kiện (Tạo, sửa, xóa), quản lý số chỗ (availableSeats).

Module	Trách nhiệm chính
Registration Module	Xử lý logic đăng ký, kiểm tra điều kiện tham gia, phối hợp với Payment.
Payment Module	Tích hợp cổng thanh toán, quản lý trạng thái giao dịch (Pending, Success, Failed).
Notification Hub	Quản lý danh sách kênh (Email, Telegram...), định dạng nội dung và gửi thông báo.

Ranh giới Business Logic & Infrastructure

- **Business Logic (Core):** Nằm trong các Service và Domain Entity. Ví dụ: Logic "Chỉ gửi vé QR sau khi thanh toán thành công" thuộc về Business.
- **Infrastructure (Hạ tầng):** Nằm ở các lớp Repository (truy cập DB) và Adapter (gọi API Telegram, SMS, Payment Gateway). Business Logic sẽ gọi Infrastructure thông qua **Interface**.

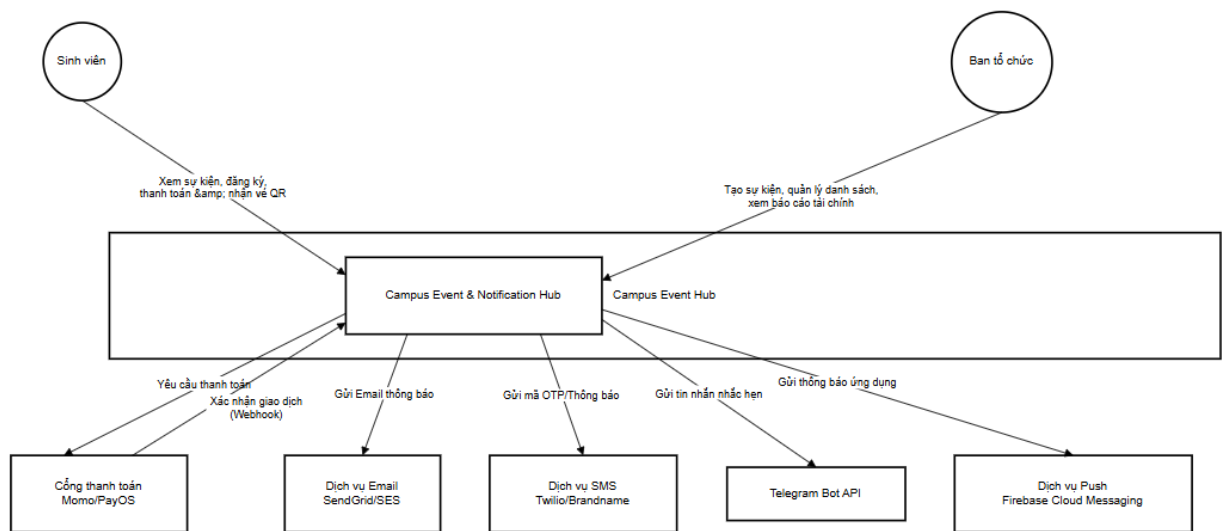
Cải thiện so với thiết kế cũ

- **Tính kết dính cao:** Sửa logic báo cáo tài chính sẽ không đụng chạm đến code gửi thông báo.
- **Dễ mở rộng:** Thêm Telegram chỉ là thêm một Adapter mới trong Notification Hub, không cần sửa hàm register.

Phản biện thiết kế:

- **Quyết định:** Sử dụng **Domain Events** (ví dụ: RegistrationSuccessEvent) để kết nối các module.
- **Phương án loại bỏ:** Gọi trực tiếp module này từ module kia (Tight coupling).
- **Lý do:** Gọi trực tiếp sẽ tạo ra sự phụ thuộc vòng. Dùng Event giúp mô-đun Đăng ký không cần biết hệ thống có gửi Telegram hay không, giúp code cực kỳ sạch.

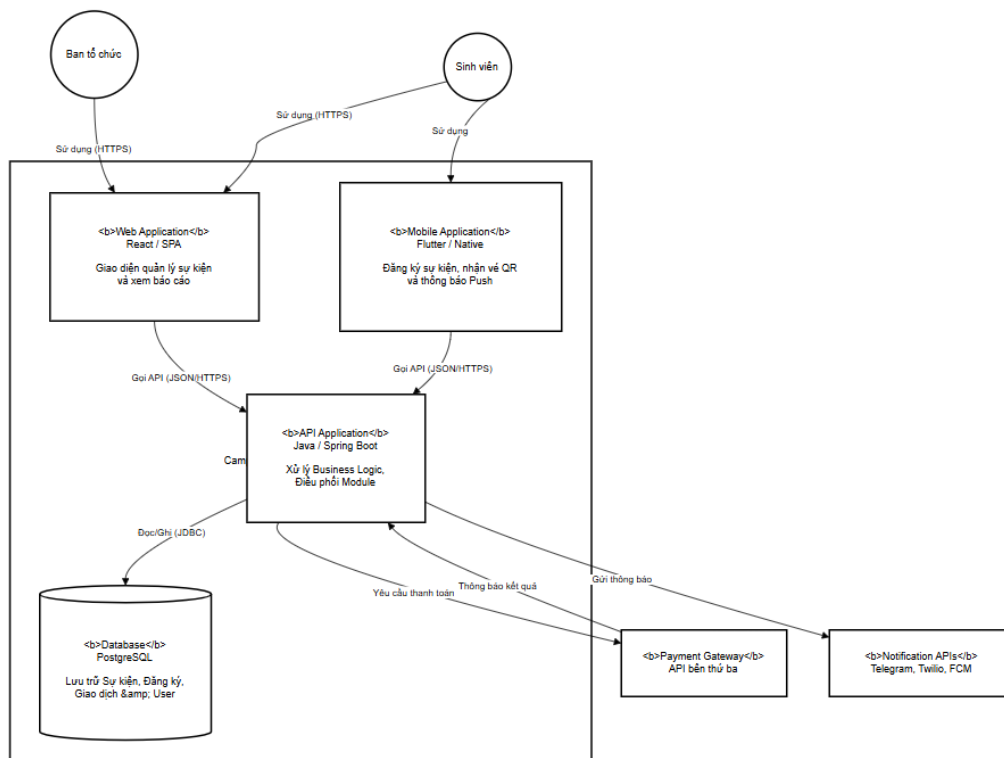
2. C4 Context Diagram



Phân tích tư duy phản biện (Câu 2)

- **Quyết định chọn:** Tôi mô hình hóa mỗi kênh thông báo (Email, SMS, Telegram, Push) như một hệ thống bên ngoài riêng biệt trong Context Diagram thay vì gom chúng lại thành một "Notification Provider".
- **Phương án đã cân nhắc:** Sử dụng một **Notification Aggregator** (một dịch vụ trung gian quản lý tất cả các kênh).
- **Lý do loại bỏ:** Ở cấp độ **Context**, việc tách rõ từng kênh giúp Ban giám hiệu và CTO thấy rõ được khả năng mở rộng (như việc thêm Telegram) mà không cần đi sâu vào chi tiết kỹ thuật bên trong. Điều này cũng giúp làm rõ các chi phí tích hợp khác nhau (SMS thì tốn phí, Telegram thì miễn phí).

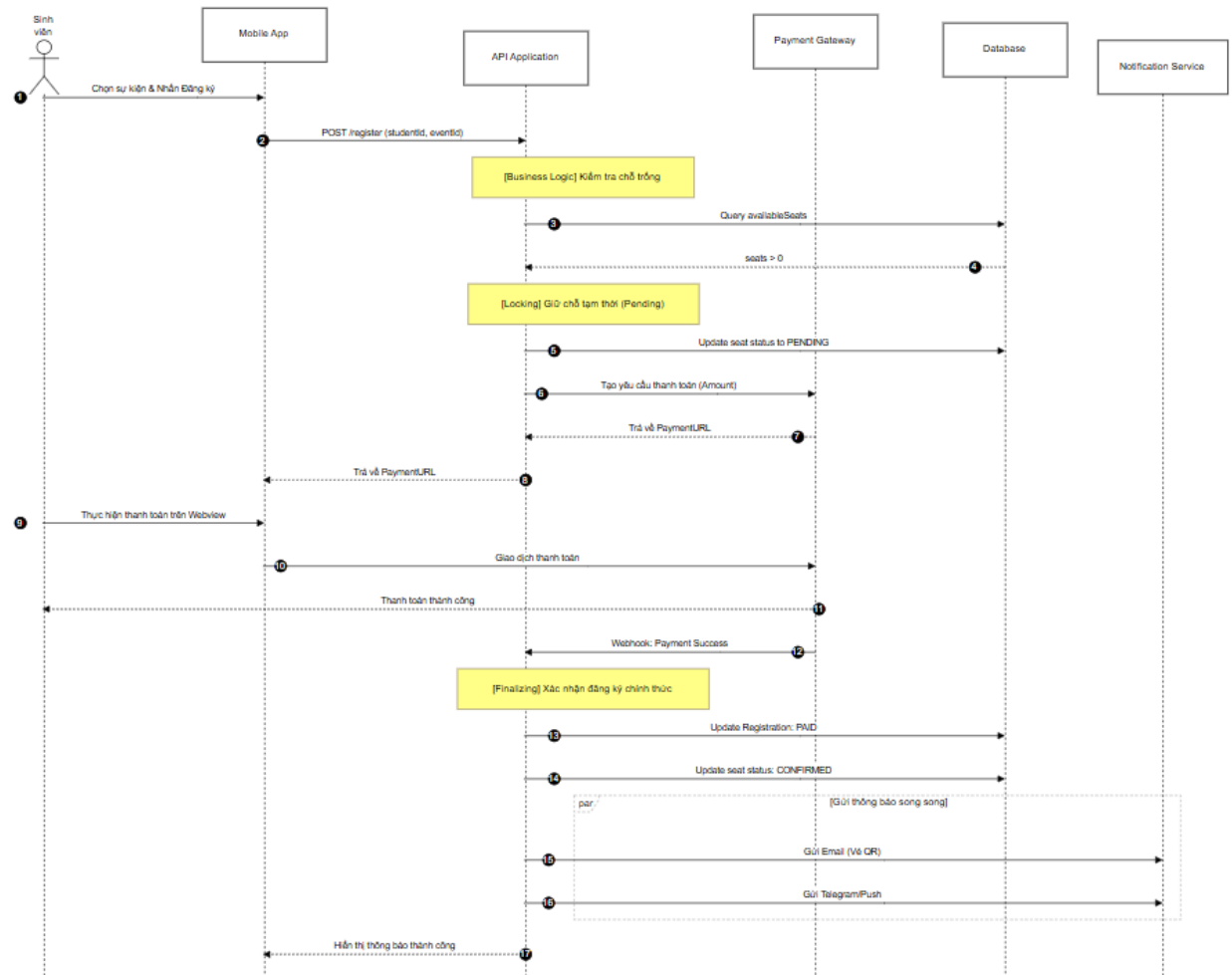
3. C4 Container Diagram



- **Quyết định chọn:** Tôi chọn kiến trúc **Modular Monolith** (tất cả các module chạy chung trong một API Application).
- **Phương án khác đã cân nhắc:** Tách thành các **Microservices** riêng biệt (Event Service, Notification Service, Payment Service).
- **Lý do loại bỏ:** Ở quy mô trường đại học, lưu lượng truy cập không đủ lớn để bù đắp cho sự phức tạp của Microservices. Việc tách service sẽ dẫn đến vấn đề "Distributed Transactions" (Giao dịch phân tán)—rất khó để đảm bảo rằng khi trừ tiền sinh viên thành công thì chỗ ngồi cũng được cập nhật chính xác nếu hai dịch vụ này nằm ở hai database khác nhau.

4. UML Sequence Diagram

Tôi chọn luồng "**Đăng ký sự kiện có thanh toán**" vì đây là luồng thể hiện rõ nhất sự phối hợp giữa các thành phần mới và cách giải quyết lỗi "giảm chỗ trước khi trả tiền" ở thiết kế cũ.



Phân tích tư duy phản biện (Câu 4)

- **Quyết định chọn:** Cơ chế **PENDING Seat**. Khi sinh viên bắt đầu thanh toán, hệ thống sẽ tạm giữ chỗ (trong khoảng 10-15 phút).
- **Phương án khác đã cân nhắc:** Chỉ giảm số lượng chỗ (availableSeats--) sau khi nhận được xác nhận thanh toán thành công từ Gateway.
- **Lý do loại bỏ:** Tránh rủi ro "vỡ trận" đăng ký. Nếu không giữ chỗ trước, có thể xảy ra trường hợp 10 sinh viên cùng thanh toán cho 1 chỗ cuối cùng. Người thanh toán chậm hơn sẽ bị mất chỗ dù đã bị trừ tiền, dẫn đến quy trình hoàn tiền (Refund) cực kỳ phức tạp và gây bức xúc cho sinh viên.

B)

1. KHÔNG tách hệ thống thành Microservices

Dù đề bài có nhiều module (Event, Payment, Notification), tôi sẽ giữ chúng trong một **Modular Monolith** (Đơn khối phân mô-đun).

- **Lý do:** Đối với quy mô một trường đại học, lưu lượng truy cập thường chỉ tăng đột biến vào một vài thời điểm (lúc mở đăng ký), nhưng tổng lượng dữ liệu không quá khổng lồ.
- **Hậu quả nếu làm:** Tách Microservices sẽ dẫn đến phải quản lý Network Latency, Distributed Tracing và đặc biệt là **Distributed Transactions**. Nếu trừ tiền sinh viên ở Service A mà cập nhật chỗ ở Service B bị lỗi, việc xử lý hoàn tiền/đồng bộ dữ liệu sẽ phức tạp gấp 10 lần so với việc dùng một Database Transaction duy nhất.

2. KHÔNG sử dụng Message Broker hạng nặng (như Kafka/RabbitMQ)

Tôi sẽ dùng **Internal Event Bus** (như Spring Events hoặc một hàng đợi đơn giản trong DB) để gửi thông báo.

- **Lý do:** Yêu cầu gửi Telegram/Email không đòi hỏi tốc độ xử lý hàng triệu tin nhắn mỗi giây. Việc cài đặt và duy trì một cụm Cluster Kafka chỉ để gửi vài ngàn thông báo mỗi ngày là cực kỳ lãng phí tài nguyên máy chủ và nhân lực vận hành.
- **Hệ quả:** Nếu dùng Kafka, bộ phận IT của trường sẽ phải cử người theo dõi, bảo trì hệ thống này. Thay vào đó, dùng logic xử lý bất đồng bộ (Async) ngay trong API giúp hệ thống vẫn mượt mà mà không tốn thêm chi phí vận hành.

3. KHÔNG áp dụng Database-per-service (Mỗi module 1 DB)

Tôi sẽ sử dụng **một Database PostgreSQL duy nhất** nhưng phân chia Schema rõ ràng.

- **Lý do:** Dữ liệu Sinh viên, Sự kiện và Thanh toán có quan hệ chặt chẽ với nhau. Việc dùng chung một Database cho phép sử dụng lệnh JOIN thần thánh và các ràng buộc khóa ngoại (Foreign Keys) để đảm bảo dữ liệu luôn đúng đắn.
- **Hệ quả:** Nếu tách DB ngay từ đầu theo kiểu Microservices, bạn sẽ không thể JOIN dữ liệu để xuất báo cáo tài chính dễ dàng. Bạn phải viết thêm một "Reporting Service" để tổng hợp dữ liệu từ nhiều nguồn – đây chính là định nghĩa của **Over-engineering**.

4. KHÔNG xây dựng Hệ thống thông báo vạn năng (Generic Notification Engine)

Tôi sẽ chỉ tạo các **Notification Strategy** cụ thể cho Email, SMS, Push và Telegram như đề bài yêu cầu.

- **Lý do:** Nhiều lập trình viên thích xây dựng một Engine có thể tự động chọn kênh, tự động retry 10 lần, hỗ trợ hàng trăm loại provider... ngay từ đầu. Tuy nhiên, đề bài chỉ yêu cầu gửi thông báo đăng ký thành công.

- **Hệ quả:** Việc xây dựng một Engine quá tổng quát (Generalization) khi chưa rõ nhu cầu thực tế sẽ làm mã nguồn trở nên cực kỳ trừu tượng và khó đọc. Hãy áp dụng **YAGNI**: Chỉ xây dựng những gì cần dùng bây giờ. Khi nào trường cần thêm 20 kênh nữa, lúc đó refactor cũng chưa muộn.

5. KHÔNG tự triển khai hạ tầng Kubernetes (K8s)

Tôi đề xuất triển khai trên **Docker Compose** hoặc các nền tảng **PaaS** (như Heroku, Render) hoặc đơn giản là một máy chủ ảo (VPS) đủ mạnh.

- **Lý do:** Kubernetes là để quản lý hàng trăm container. Với một hệ thống Campus chỉ gồm 1 API và 1 DB, K8s sẽ làm tăng chi phí hạ tầng lên gấp 3-4 lần và đòi hỏi chuyên gia DevOps vận hành.
- **Hệ quả:** Dùng Docker Compose đơn giản giúp lập trình viên có thể tự quản lý và triển khai (Deploy) trong vài phút, đúng tinh thần "Keep It Simple".

Câu 3 (2.0 điểm) — Chọn pattern phù hợp và bác bỏ một pattern không nên dùng

Chọn **2 design patterns phù hợp** cho bài toán, rồi **bác bỏ 1 pattern không nên dùng ở thời điểm này**.

Với mỗi pattern được chọn, em phải trả lời:

1. Pattern này giải quyết đúng vấn đề nào trong case?
2. Nó tác động vào phần nào của thiết kế mới?
3. Nó hỗ trợ nguyên lý nào trong SOLID?
4. Nó cải thiện hệ thống ở điểm nào?
5. Nó tạo thêm chi phí hay độ phức tạp gì?

Với pattern bị bác bỏ, em phải trả lời:

1. Tên pattern
2. Vì sao nó **có vẻ hợp lý lúc đầu**
3. Vì sao **không nên dùng lúc này**
4. Trong điều kiện nào tương lai nó có thể trở nên hợp lý

Ràng buộc bắt buộc

- Không được chỉ viết kiểu: “Em chọn Strategy vì dễ mở rộng”
- Phải chỉ rõ mở rộng **cái gì**, thay đổi **ở đâu**, và **vì sao chưa cần pattern khác**

- Nếu chọn 2 pattern, phải nói rõ **ranh giới** giữa chúng

Giải

Để giải quyết triệt để các vấn đề của thiết kế cũ mà vẫn giữ cho hệ thống tinh gọn, tôi chọn **Observer** và **Strategy**.

1. Pattern 1: Observer (Mẫu người quan sát)

- **Vấn đề giải quyết:** Sự phụ thuộc quá mức (Tight Coupling) giữa logic đăng ký và các kênh thông báo. Thiết kế cũ buộc hàm register phải biết về EmailSender, SmsSender,...
- **Tác động vào thiết kế:** Nằm ở ranh giới giữa Registration Module (Subject/Publisher) và Notification Module (Observers/Subscribers). Khi sinh viên đăng ký thành công, một sự kiện StudentRegistered sẽ được bắn ra.
- **Hỗ trợ SOLID: * OCP (Open/Closed):** Khi thêm kênh **Telegram**, ta chỉ cần tạo một TelegramObserver mới và đăng ký nó vào hệ thống mà không cần sửa một dòng code nào trong logic đăng ký.
 - **SRP (Single Responsibility):** Lớp đăng ký chỉ tập trung vào nghiệp vụ đăng ký; việc gửi thông báo là trách nhiệm của các lớp Observer.
- **Cải thiện hệ thống:** Giúp hệ thống linh hoạt hơn. Bạn có thể bật/tắt các kênh thông báo theo cấu hình của từng sự kiện mà không làm rối mã nguồn nghiệp vụ.
- **Chi phí/Độ phức tạp:** Tạo thêm một cơ chế quản lý danh sách Observer (Event Bus) bên trong ứng dụng. Luồng thực thi không còn là đường thẳng (Linear), gây khó khăn nhẹ cho người mới khi muốn tìm xem cái gì xảy ra sau khi đăng ký.

2. Pattern 2: Strategy (Mẫu chiến lược)

- **Vấn đề giải quyết:** Sự phức tạp của yêu cầu **Một số sự kiện cần thanh toán**. Mỗi loại sự kiện (Miễn phí, Trả phí qua Momo, Trả phí qua thẻ) có một thuật toán xử lý khác nhau.
- **Tác động vào thiết kế:** Nằm trong Payment Module. RegistrationService sẽ nhận vào một IPaymentStrategy để xử lý giao dịch.
- **Hỗ trợ SOLID: * DIP (Dependency Inversion):** Mô-đun đăng ký mức cao phụ thuộc vào IPaymentStrategy (Trừu tượng) chứ không phụ thuộc vào lớp MomoPayment hay FreePayment cụ thể.

- **Cải thiện hệ thống:** Loại bỏ hoàn toàn các khối if-else lồng nhau để kiểm tra loại thanh toán. Code trở nên "sạch" và dễ đọc. Việc kiểm thử (Unit Test) cho từng phương thức thanh toán trở nên độc lập và dễ dàng.
- **Chi phí/Độ phức tạp:** Tăng số lượng class (mỗi phương thức thanh toán là một class). Nếu hệ thống chỉ có đúng 1 loại thanh toán duy nhất mãi mãi, việc dùng Strategy sẽ bị coi là over-engineering.

Ranh giới giữa Observer và Strategy trong hệ thống này:

- **Strategy** được dùng **TRƯỚC** khi hoàn tất đăng ký (để xử lý tiền bạc, trả về kết quả thành công/thất bại).
- **Observer** được dùng **SAU** khi đăng ký thành công (để thông báo cho các bên liên quan như Gửi mail, Gửi Telegram, Cập nhật dashboard quản lý).

3. Pattern bị bác bỏ: Chain of Responsibility (Chuỗi trách nhiệm)

- **Vì sao có vẻ hợp lý lúc đầu:** Nhiều người nghĩ rằng có thể dùng chuỗi này để xử lý thông báo: chuyển yêu cầu đăng ký qua Email -> SMS -> Telegram. Mỗi mắt xích sẽ quyết định có gửi hay không.
- **Vì sao không nên dùng lúc này:** 1. **Sai mục đích:** Chain of Responsibility thường dùng khi chỉ cần **một** đối tượng trong chuỗi xử lý yêu cầu (ví dụ: duyệt đơn từ cấp thấp lên cấp cao). Trong khi đó, hệ thống Campus cần gửi **đồng thời** trên nhiều kênh. 2. **Over-engineering:** Việc xây dựng các lớp Handler nối tiếp nhau làm code trở nên rắc rối vô ích cho một nhu cầu phát sóng (Broadcast) đơn giản. Nó khiến việc kiểm soát thứ tự và trạng thái gửi của từng kênh trở nên cực kỳ khó khăn.
- **Điều kiện hợp lý trong tương lai:** Nếu trường đại học có quy trình **phê duyệt sự kiện phức tạp** (ví dụ: Sự kiện CLB tạo ra -> Bí thư Đoàn duyệt -> Phòng Công tác sinh viên duyệt -> Ban Giám hiệu ký). Khi đó, mỗi cấp là một mắt xích, nếu cấp dưới không duyệt thì yêu cầu không được chuyển lên cấp trên. Đây mới là lúc Chain of Responsibility tỏa sáng.